

GreenTrails — Dependability Report

Gabriele Di Stefano
g.distefano10@studenti.unisa.it

Roberta Galluzzo
r.galluzzo3@studenti.unisa.it

Abstract—<https://github.com/gab-ds/GreenTrails>

Il presente documento fornisce una sintesi delle attività di manutenzione perfettiva dell'applicativo web GreenTrails dal punto di vista della Software Dependability, illustrando le metodologie di verifica, gli strumenti di analisi e i risultati ottenuti per ciascun attributo di affidabilità: test unitari e di integrazione, code coverage (JaCoCo), mutation testing (Pitest), analisi statica (Checkstyle, SonarQube con Creedengo), vulnerability scanning (Snyk) e secret scanning (GitGuardian).

Index Terms—Dependability, Software Reliability, Mutation Testing, JaCoCo, Pitest, Security

I. INTRODUZIONE

Il presente documento fornisce una sintesi delle attività di manutenzione perfettiva dell'applicativo web GreenTrails dal punto di vista della Software Dependability. L'applicativo è già oggetto di manutenzione evolutiva e perfettiva, rispettivamente per i corsi di Ingegneria del Software: Tecniche Avanzate e Sustainable Software Engineering.

Il documento illustra le ottimizzazioni tecniche apportate, con una spiegazione del razionale alla base di ciascun intervento e dei relativi requisiti. Include la descrizione tecnica di ogni intervento — obiettivi, specifiche, problemi riscontrati e risultati — e in chiusura riporta sintesi delle modifiche, problematiche residue e considerazioni finali.

Le attività di manutenzione perfettiva svolte nell'ambito della Software Dependability hanno riguardato:

- **Containerizzazione:** realizzazione di immagini Docker multi-stage per backend e frontend, con configurazioni multi-ambiente (sviluppo, produzione, test) orchestrate tramite Docker Compose.
- **CI/CD:** creazione di workflow GitHub Actions per test automatizzati (`mvn test`, `mvn verify`), analisi di copertura (JaCoCo), mutation testing (Pitest), analisi statica (Checkstyle, SonarQube), e build e pubblicazione di immagini Docker su Docker Hub.
- **Security hardening:** configurazione di autenticazione HTTP Basic con `BCryptPasswordEncoder`, autorizzazione basata su ruoli, CORS, vulnerability scanning (Snyk), secret scanning (GitGuardian) e analisi delle vulnerabilità del frontend (7 vulnerabilità CWE/OWASP identificate e documentate).
- **DevSecOps:** integrazione di scansioni di sicurezza nei workflow CI/CD per garantire verifiche continue su dipendenze, segreti e qualità del codice.
- **Testing con coverage e mutation testing:** suite di test unitari e di integrazione con JUnit 5 e Mockito, misurata con JaCoCo (soglia minima 80%) e rafforzata con Pitest

per valutare l'efficacia reale dei test nel rilevare errori introdotti artificialmente.

A. Contesto

a) *Scopo dell'applicazione:*

L'obiettivo principale dell'applicazione è quello di agevolare la progettazione di itinerari ecosostenibili, offrendo agli utenti uno strumento intuitivo e completo per organizzare viaggi rispettosi dell'ambiente. La piattaforma si propone di combinare la facilità d'uso con un approccio basato su criteri di sostenibilità.

b) *Ruoli definiti:*

All'interno dell'applicazione sono stati individuati tre attori, ciascuno con responsabilità ben distinte:

- **il Visitatore**, che rappresenta l'utente finale che accede alla piattaforma per consultare e prenotare le attività disponibili;
- **il Gestore Attività**, ovvero la figura incaricata di creare, pubblicare e amministrare le proprie attività sul portale;
- **l'Amministratore**, il quale detiene i poteri di gestione del sito, tra cui la possibilità di modificare la percentuale di commissione applicata ai guadagni derivanti dalle prenotazioni.

c) *Valori di ecosostenibilità:*

Un concetto cruciale per il funzionamento della piattaforma è quello dei **valori di ecosostenibilità**: una lista strutturata di buone pratiche ambientali che ogni attività può adottare e dichiarare. Essi vengono registrati nella piattaforma e fungono da criteri di valutazione, consentendo sia ai visitatori sia all'algoritmo di classificare le attività in base al loro impegno ambientale.

d) *Preferenze del visitatore:*

Le preferenze espresse dal visitatore costituiscono un elemento fondamentale per la personalizzazione dell'esperienza; esse possono riguardare diversi aspetti, quali località preferita (es. mare, montagna, città), tipologia di attività preferita (es. ristoranti, all'aperto) e così via. Queste informazioni vengono elaborate dal modulo di intelligenza artificiale integrato nella piattaforma (un algoritmo genetico) che, utilizzando sia i dati relativi ai valori di ecosostenibilità delle attività sia le preferenze dell'utente, genera un itinerario su misura ottimizzato per minimizzare la distanza percorsa.

II. ARCHITETTURA DEL SISTEMA

Il backend segue un'architettura a **layers** propria di Spring: Presentation Layer (controller REST MVC), Service Layer (logica di business), Persistence Layer (repository JPA con MySQL in produzione e H2 in test), Security Layer (Spring Security). Il progetto è organizzato per domini funzionali

in package dedicati: gestioneutenze, gestioneattivit , gestioneitinerari, gestioneprenotazioni, gestioneicerca, gestione-segnalazioni, gestioneupload e utils.

A. Stack Tecnologico

Il backend   sviluppato in **Java 21** con **Spring Boot 3.5.16** (aggiornato da 3.2.1), moduli Web MVC, Data JPA, Security (HTTP Basic + BCrypt), Actuator, Validation. Dipendenze gestite con Maven, profiling per ambienti dev/prod/test. Database MySQL 8 (produzione) / H2 embedded (test).

Il frontend   stato oggetto di un'importante migrazione durante la manutenzione evolutiva:

- **Prima:** Angular 14 con Angular Material 13, Bootstrap 5, Karma + Jasmine per i test, servito tramite Nginx su porta 4200.
- **Dopo:** Nuxt 4 (basato su Vue 3) con TailwindCSS 4, FormKit, Pinia per la gestione dello stato, Bun come package manager, ESLint per il linting, vue-tsc per il type checking, Vitest per i test, servito tramite Node.js 22 Alpine su porta 3000 (mappata sulla 9000 in Docker).

B. Componenti Principali

- **Sicurezza:** HTTP Basic Authentication, BCryptPasswordEncoder (work factor 10, 130 ms per hash), controllo accessi basato su ruoli (VISITATORE, GESTORE_ATTIVITA, AMMINISTRATORE), CORS configurato per frontend su localhost:9000 e :3000.
- **Servizi Core:** GestioneUtenzeServiceImpl (UserDetailsService), Attivit ServiceImpl (CRUD e filtri attivit ), ItinerariServiceImpl (pianificazione itinerari con adattatore AI in stub), PrenotazioneAlloggioServiceImpl / PrenotazioneAttivit TuristicaServiceImpl (ciclo di vita prenotazioni), RicercaServiceImpl (filtri spaziali).
- **Utility:** CorsConfig, ResponseGenerator.

C. Infrastruttura e Distribuzione

La configurazione iniziale utilizzava immagini Docker pubbliche upstream (eclipse-temurin:21 per backend, oven/bun:1 + node:22-alpine per frontend) senza alcun controllo di sicurezza sulle immagini di base.

Nel corso dell'hardening CI/CD sono state applicate le seguenti ottimizzazioni:

- **Immagini hardened:** le immagini pubbliche sono state sostituite con le corrispondenti versioni dhi.io (Docker Hardened Images):
 - 1) Backend builder: dhi.io/eclipse-temurin:21-jdk-alpine-dev
 - 2) Backend runtime: dhi.io/eclipse-temurin:21-alpine
 - 3) Frontend builder: dhi.io/bun:1-alpine-dev
 - 4) Frontend runtime: dhi.io/node:22-alpine3.23

Tali immagini includono vulnerability scanning integrato e riducono la superficie d'attacco rispetto alle upstream. Ogni immagine   pinnata per SHA nei Dockerfile per garantire build riproducibili; Dependabot aggiorna automaticamente i digest alla scansione settimanale.

- **Hadolint:** linting statico dei Dockerfile sia in CI/CD (tramite hadolint-action nei workflow backend.yml e frontend.yml) sia localmente come pre-commit hook, con soglia warning per garantire best practice nella scrittura dei Dockerfile.

La configurazione multi-ambiente rimane invariata: sviluppo (docker-compose.yml orchestra backend, frontend Nuxt 4, MySQL) e produzione (docker-compose.prod.yml con policy di riavvio e limiti risorse). Health check su / actuator/health per monitoraggio della disponibilit  del backend.

D. Buildabilit 

L'applicazione   buildabile sia in CI/CD sia localmente:

- **CI/CD — configurazione iniziale:** il workflow backend.yml eseguiva i job separati test e coverage come prerequisiti per il job docker; il workflow frontend.yml eseguiva lint, typecheck e test come job distinti. Non erano presenti controlli di supply chain security.
- **CI/CD — hardening applicato:**
 - 1) **Backend:** i job test e coverage sono unificati in un unico job test che esegue Checkstyle, mvn test, JaCoCo e Pitest in sequenza, con timeout massimo di 45 minuti. Un nuovo job deploy-reports pubblica i report su GitHub Pages (con solo permesso contents: write).
 - a) **Reporting coverage in CI:** due script Python (scripts/jacoco_coverage.py e scripts/pitest_coverage.py) leggono i report XML generati (jacoco.xml, mutations.xml) e stampano un riepilogo del coverage. Con --markdownify l'output   in Markdown (tabelle ed emoji) e viene mostrato nello step summary e come commento sulla pull request (via thollander/actions-comment-pull-request); senza flag, una lista in testo semplice per l'uso locale da CLI. Le mutazioni sopravvissute di Pitest sono raggruppate in una sezione collapsabile <details>. Gli stessi script implementano i gate --fail-below 80 (JaCoCo linee e Pitest) sulle pull request, sostituendo la precedente action PavanMudigonda/jacoco-reporter.
 - b) **Report conservati anche a test falliti:** gli step di pubblicazione (badge, report, summary, deploy) usano if: !cancelled() invece di continue-on-error, quindi il riepilogo e i report vengono generati anche quando i test falliscono; la pubblicazione su Docker Hub resta comunque bloccata a cascata perch  il job docker-push dipende dal job test via needs.

- c) **Test report:** l'azione `scacap/action-surefire-report v1` è stata migrata al successore `ScalableCapital/action-surefire-report v2.0.5`, che pubblica l'esito dei test Surefire sulla PR.
 - d) **Docker:** il job `docker` ora esegue la build dell'immagine anche su pull request (`push: false`), non solo su `main`. Un nuovo job `docker-push`, separato, pubblica l'immagine su Docker Hub solo su push a `main`, vincolato all'`environment: production` — protetto da **required reviewers**: la pubblicazione richiede una review manuale prima dell'esecuzione — e genera l'SBOM (`sbom: true`) con `provenance: mode=max`.
- 2) **Fix del deploy dei report (bug critico upload-artifact):** `actions/upload-artifact@v4` taglia dalle cartelle caricate il “least common ancestor” dei path, quindi con i tre path multipli (`target/site/jacoco/`, `target/reports/`, `target/pit-reports/`) l'artifact arrivava al job `deploy-reports` senza il prefisso `backend/` `target/`, facendo fallire la pubblicazione su GitHub Pages con `cp: cannot stat './backend/target/reports/*'`. Il job `test` ora assembla esplicitamente la cartella `public/` (con `surefire/`, `jacoco/`, `pit/` già in posizione) e carica un singolo path `./public/`; `deploy-reports` scarica l'artifact e pubblica senza passaggi di copia, rendendo la struttura del deploy esplicita e indipendente dalla semantica implicita dell'LCA.
- 3) **Frontend:** `lint`, `typecheck` e `test` sono unificati in un unico job `gate` (`bun run gate`) con timeout di 10 minuti. La stessa struttura a due job di Docker del backend vale anche per l'immagine frontend: build su pull request (`push: false`) e `docker-push` solo su push a `main` con `environment: production`, SBOM e `provenance`.
- 4) **Hadolint:** linting dei Dockerfile aggiunto in entrambi i workflow.
- 5) **Caching multi-livello delle build Docker:** i job `docker` riusano i layer delle build precedenti tramite la cache di GitHub Actions (`cache-from/cache-to: type=gha` con `mode=max`, che include anche lo stage builder). Inoltre i `RUN --mount=type=cache` di `BuildKit` (`/root/.m2` per Maven, la cache di `bun install` per il frontend) persistono le dipendenze scaricate tra build diverse: quando cambia solo `pom.xml/bun.lock` si scarica solo il delta, non l'intero set di dipendenze.

L'azione `buildkit-cache-dance`, infine, garantisce che la mount cache venga persistita su GitHub, estraendo e iniettandone il contenuto.

- 6) **Supply chain security:** tutte le GitHub Actions sono pinnate per SHA con commento di versione; ogni workflow dichiara `permissions: {}` come permessi di default, e concede solo i permessi strettamente necessari per job; ogni job definisce un timeout-massimo per prevenire esecuzioni troppo lunghe. Le immagini pubblicate su Docker Hub includono l'SBOM (`sbom: true`) e le attestazioni di provenance (`provenance: mode=max`).

- **Locale:** Maven Wrapper (`./mvnw`) disponibile, comandi standard `./mvnw compile`, `./mvnw package`, `./mvnw verify`. Docker multi-stage per build containerizzato da zero.

III. VERIFICA DELLA DEPENDABILITY

Il progetto adotta una strategia di verifica a più livelli per garantire ciascun attributo della dependability.

A. Test Unitari e di Integrazione

- **Test Unitari:** 30 classi di test con Mockito per service layer, che coprono isolatamente la logica di business di ogni dominio.
- **Test di Integrazione:** 12 classi con `@SpringBootTest` e `@AutoConfigureMockMvc` per i controller REST, che verificano il corretto funzionamento dell'intero stack HTTP.
- **Suite totale:** 451 test, di cui 8 temporaneamente esclusi (relativi all'AI adapter in attesa di manutenzione evolutiva).

a) JaCoCo — Code Coverage:

JaCoCo (Java Code Coverage) misura la percentuale di codice sorgente eseguita durante i test, analizzando copertura di linee, rami, metodi e classi. Nel progetto è integrato come plugin Maven con soglia minima dell'80%. Inizialmente la soglia era applicata direttamente nella build: se la copertura scendeva sotto tale valore, la build falliva. Nel corso della manutenzione il controllo è stato spostato in CI/CD tramite lo script `scripts/jacoco_coverage.py --fail-below 80`, eseguito sulle pull request al posto della precedente action `PavanMudigonda/jacoco-reporter`.

Risultati: la copertura attuale si attesta al 97,5% sulle linee, 97,2% sui rami e 100% sulle classi (30/30), con i service layer al 100%. Le esclusioni `agent/report` sono state allineate: `entity`, `enum`, `utility`, eccezioni di gestione `upload`, `security` e `BackendApplication`, classi senza logica di business significativa.

b) Pitest — Mutation Testing:

Pitest è un framework di mutation testing che valuta l'efficacia dei test introducendo sistematicamente piccole modifiche (mutazioni) nel codice sorgente e verificando se i test esistenti le rilevano. Una mutazione “sopravvissuta” indica una lacuna nella suite di test.

Nel progetto è integrato come plugin Maven con versione 1.22.1, configurato per escludere entity, enum, classi di configurazione e l'AI adapter (esclusione temporanea). Il test viene eseguito in fase `verify` con soglia minima dell'80% di mutation coverage, verificata in CI/CD tramite lo script `scripts/pitest_coverage.py --fail-below 80` sulle pull request.

Risultati: la mutation coverage si attesta all'89,6% (583 mutazioni uccise su 651 valide), con 68 mutazioni sopravvissute riportate nella sezione collapsabile dei commenti PR. I report HTML vengono generati nella directory `target/pit-reports/`.

c) Checkstyle — Stile del Codice:

Checkstyle è uno strumento di analisi statica che verifica la conformità del codice alle convenzioni di stile definite. Il progetto adotta le regole **Google Java Style** tramite il file `google_checks.xml`, con una sola soppressione per i commenti Javadoc (considerati ridondanti data l'auto-documentazione del codice tramite nomi chiari e Lombok).

Risultati: 0 violazioni delle regole Google Java Style su tutto il codice backend. Il check viene eseguito in fase `verify` e in CI/CD tramite GitHub Actions.

d) SonarQube e Creedengo — Analisi Statica:

SonarQube 9.9.8 è stato installato su Docker per l'analisi statica del codice backend (4.352 linee). Il plugin **Creedengo** 2.0.0 (ex ecoCode) estende l'analisi con 15 regole specifiche per l'efficienza energetica e la qualità del codice Java.

Risultati:

- **Bug:** 0 — **Vulnerabilità:** 0 — **Code Smells:** 3
- **Debito tecnico:** 150 minuti (0,1% del costo di sviluppo)
- **Duplicazioni:** 5,1%
- **Rating:** affidabilità A, sicurezza A, manutenibilità A

I 3 code smell appartengono alla regola **GCI1 — Avoid Spring repository call in loop or stream**, presenti in:

File	Linea	Descrizione
ItinerariStubAdapter.java	48	Repository call in stream
ItinerariStubAdapter.java	62	Repository call in stream
RicercaServiceImpl.java	36	Repository call in stream

Tutti con severità MINOR e debito stimato di 50 minuti ciascuno.

B. JML e OpenJML — Specifica Formale e Static Checking

Java Modeling Language (JML) è un linguaggio di specifica formale che permette di definire contratti precisi sui metodi Java tramite annotazioni: precondizioni (`@ requires`), postcondizioni (`@ ensures`), effetti collaterali (`@ assignable`) e invarianti di classe (`@ invariant`). **OpenJML** 0.21.0 è lo strumento di verifica che esegue l'Extended Static Checking (ESC) su tali contratti, dimostrando automaticamente la correttezza o segnalando potenziali violazioni.

a) Annotazioni JML introdotte:

Le annotazioni JML sono state introdotte in tutti i Service del backend, coprendo i 7 moduli funzionali (utenze, attività, itinerari, prenotazioni, ricerca, segnalazioni, upload) più le Entity con dichiarazioni `@ nullable_by_default`.

La verifica è organizzata per moduli, ciascuno con un profilo Maven dedicato (`./mvnw verify -P openjml-{modulo}`).

b) Risultati preliminari:

Modulo	Failures	Warnings	Errori interni	Stato
Utenze	16 -> 6	4	0	Completato
Attività	37 -> 32	5	0	Completato
Itinerari	33 -> 22	2	0	Completato
Prenotazioni	28 -> crash	10	1	Crash OpenJML
Ricerca	42 -> 42	0	0	Nessuna riduzione
Segnalazioni	9 -> 6	0	0	Completato
Upload	93 -> 80	11	0	Completato

Le correzioni hanno ridotto i failure da 258 a circa 188 (27%) nei 6 moduli verificabili. Il modulo Prenotazioni crasha ancora per un bug interno di OpenJML 0.21.0. La maggior parte dei failure residui e' causata da tre fattori strutturali non risolvibili:

- `@nullable_by_default` su tutte le classi rende ogni valore nullable
- Lombok genera costruttori/getter senza annotazioni JML
- Le librerie standard (String, Math, Path, repository) non hanno specifiche verificabili

Va notato che la maggior parte dei failure e' imputabile all'uso combinato di Lombok e Spring Data JPA. In un progetto senza questi framework – con costruttori scritti a mano, repository JDBC tradizionali e dependency injection esplicita – OpenJML verificherebbe il codice con molti meno falsi positivi, probabilmente avvicinandosi a zero failure. Questo non e' un limite di OpenJML in se', ma una conseguenza dell'attrito tra specifiche formali e framework moderni che generano o invocano codice dinamicamente.

Alla luce di questi risultati, si puo' concludere che per questo specifico progetto l'integrazione di OpenJML si e' rivelata piu' dannosa che utile: lo sforzo di annotazione e manutenzione delle specifiche non ha prodotto benefici proporzionali, perche' la stragrande maggioranza dei failure residui e' costituita da falsi positivi generati da Lombok e Spring, non da bug reali nel codice applicativo.

IV. ANALISI DELLA SICUREZZA

La sicurezza informatica è uno degli attributi fondamentali della dependability. Il progetto adotta un approccio a più livelli: configurazione di autenticazione e autorizzazione, vulnerability scanning delle dipendenze e secret scanning del codice.

A. Spring Security — Autenticazione e Autorizzazione

La configurazione di sicurezza è gestita centralmente in `SecurityConfig.java`, che definisce:

- **Autenticazione HTTP Basic:** ogni richiesta autenticata include username e password nell'header `Authorization`, codificati in Base64. Le password sono hashate con BCrypt (work factor 10, 130 ms per hash).
- **Autorizzazione basata su ruoli:** tre ruoli distinti (VISITATORE, GESTORE_ATTIVITA, AMMINISTRATORE) con permessi granulari su ogni endpoint REST.
- **CORS configurato:** whitelist di origini consentite (localhost:4200, localhost:9000, frontend), metodi HTTP e header.
- **CSRF disabilitato:** scelta consapevole per un'API REST stateless che utilizza solo autenticazione HTTP Basic, dove il CSRF non è applicabile.
- **Endpoint Actuator:** `/actuator/health` pubblico per monitoring; altri endpoint Actuator riservati ad utenti autenticati.

Esempio di controllo accessi:

- GET `/api/attivita/all` — pubblico (permittAll)
- POST `/api/attivita` — solo GESTORE_ATTIVITA
- DELETE `/api/segnalazioni/*` — solo AMMINISTRATORE
- `/api/itinerari/**` — solo VISITATORE

B. Snyk — Vulnerability Scanning

Snyk è uno strumento di Software Composition Analysis (SCA) che analizza le dipendenze open-source del progetto per identificare vulnerabilità note (CVE). Il backend Java utilizza Maven, e Snyk scansiona automaticamente il file `pom.xml` e il relativo albero delle dipendenze transitivo.

Snyk è integrata come GitHub App esterna, configurata per scansionare automaticamente l'intero repository a ogni push e pull request. L'analisi viene eseguita sia sul backend (`pom.xml`, dipendenze Maven) sia sul frontend (`package.json`, `bun.lock`, dipendenze npm).

Risultati: la scansione non ha rilevato vulnerabilità ad alta o media severità nelle dipendenze del backend al momento dell'analisi. Durante scansioni precedenti, Snyk aveva segnalato vulnerabilità su `esbuild` (frontend), `spring-boot-starter-parent`, `checkstyle`, `maven-checkstyle-plugin` (backend) e `eclipse-temurin:21-ubi10-minimal` (Docker image backend). Le vulnerabilità sulle dipendenze Maven e npm sono state risolte aggiornando alle versioni correnti: `spring-boot-starter-parent` 3.2.1 → 3.5.16, `checkstyle` 10.12.7 → 10.25.0, `maven-checkstyle-plugin` 3.3.1

→ 3.6.0, e aggiungendo un override per `esbuild` in `package.json` (`"esbuild": "^0.28.1"`). La vulnerabilità sull'immagine Docker di base `eclipse-temurin:21-ubi10-minimal` non è invece risolvibile: la remediation proposta da Snyk (`eclipse-temurin:21.0.11_10-jre-alpine-3.23`) è una JRE, mentre il progetto richiede un JDK per la build multi-stage.

C. GitGuardian — Secret Scanning

GitGuardian (tramite la CLI `ggshield`) esegue la scansione del repository alla ricerca di segreti accidentalmente committati: API key, password, token, certificati, e altre informazioni sensibili.

GitGuardian è integrata come GitHub App esterna, configurata per scansionare automaticamente ogni push e pull request alla ricerca di segreti accidentalmente committati. È inoltre presente un hook pre-commit locale (`gitleaks`) per il rilevamento prima del commit.

Risultati: nessun segreto rilevato nel codice del backend al momento della scansione.

D. CI/CD — Sicurezza automatizzata

La sicurezza è garantita da una combinazione di servizi esterni e una pipeline CI/CD (sia locale che remota), che hanno subito un progressivo hardening:

- **Dependabot — configurazione iniziale:** aggiornamento settimanale di GitHub Actions (hash pinning), Maven e npm.
- **Dependabot — hardening applicato:**
 - 1) Aggiunto il registro `dhi.io` per immagini Docker hardened.
 - 2) Ignorati major update per `eclipse-temurin` e `node` per garantire stabilità LTS in ambiente containerizzato.
- **SHA pinning:** tutte le GitHub Actions nei workflow sono ora pinnate per SHA, accompagnato dal pinning automatico di Dependabot con un controllo esplicito e versionato. Anche le immagini Docker nei Dockerfile sono pinnate per SHA, con Dependabot incaricato di aggiornare i digest settimanalmente.
- **Privilegi minimi:** entrambi i workflow dichiarano `permissions: {}`; ogni job concede solo i permessi strettamente necessari (es. `contents: read` per test e gate, `contents: write` solo per `deploy-reports` su gh-pages, `checks: write` e `pull-requests: write` per pubblicare i report di test su PR).
- **Timeout massimi:** ogni job definisce un timeout-massimo esplicito (45 min backend, 10 min frontend, 3 min deploy, 10 min docker) per prevenire job troppo lunghi e consumo eccessivo di risorse.
- **Hadolint:** linting statico dei Dockerfile inserito nei workflow CI con soglia warning.
- **Gitleaks** (pre-commit hook): hook locale che previene il commit di segreti prima che raggiungano il repository remoto, affiancando la scansione automatica di GitGuardian.

- **Snyk** (GitHub App): scansione automatica delle vulnerabilità nelle dipendenze a ogni push e pull request su main.
- **GitGuardian** (GitHub App): scansione automatica di segreti sull'intero repository a ogni push e PR.
- **SonarQube** (servizio esterno): analisi statica di sicurezza e qualità del codice, eseguita su Docker con il plugin Creedengo per regole di efficienza energetica.
- **CodeCov**: l'action codecov/codecov-action (pinnata per SHA) carica a ogni run del job test del backend, tramite il token CODECOV_TOKEN configurato nei secret del repository, due tipi di report: la copertura JaCoCo (target/site/jacoco/jacoco.xml, report_type: coverage, caricata solo a report generati) e i risultati dei test Surefire (target/surefire-reports/TEST-*.xml, report_type: test_results per le Test Analytics). L'upload dei test_results usa if: always(): viene eseguito anche in caso di fallimento dei test, come raccomandato da CodeCov, per garantire la visibilità dei fallimenti in Test Analytics. CodeCov fornisce una visione storica e per-PR della copertura del backend (97,5% medio sulle linee) e delle performance/fallimenti dei test, complementare ai report generati in CI.
- **Branch protection su `main`**: protezione server-side attiva sul repository che impone il passaggio obbligatorio da pull request per ogni modifica a main — nessun commit diretto è consentito — con almeno una review umana prima del merge, blocca i force push e revoca le approvazioni stale a ogni nuovo commit. Il flusso di code review è obbligatorio e alternato tra i due manutentori: chi scrive una modifica non può approvare la propria pull request, quindi la review è demandata all'altro reviewer e il ruolo si inverte a ogni modifica (es. io pusho, Bob revisiona; Bob pusha, io revisiono). I **required status checks** sono stati volutamente **non** abilitati: i workflow CI sono filtrati per path (backend/** e frontend/**), e imporre il check meccanicamente bloccherebbe in modo irreversibile le pull request che toccano solo documentazione, configurazione o workflow, perché i check non riportati da workflow skippati restano nello stato "Expected — Waiting for status to be reported". La scelta di demandare il giudizio finale a un reviewer umano è quindi consapevole: la protezione è garantita a livello di server, mentre l'esito visibile dei check CI (verde/rosso) guida la decisione dell'approvatore senza vincolare il merge in modo meccanico.

E. DevSecOps — Pre-commit Hooks

I pre-commit hook costituiscono la prima linea di difesa nel workflow, bloccando commit non conformi prima che raggiungano il repository remoto o attivino la pipeline CI/CD.

Il progetto utilizza pre-commit con i seguenti hook, configurati in .pre-commit-config.yaml:

Hook	Cosa verifica	Attivazione
------	---------------	-------------

Checkstyle	Conformità Google Java Style su tutto il backend	Staged .java in backend/
Bun Lint	ESLint sui file staged del frontend (JS/TS/Vue)	Staged .js/.ts/.vue in frontend/
Bun Typecheck	vue-tsc type checking full-project	Qualunque file in frontend/
Hadolint	Best practice di container security nei Dockerfile	Modifiche a Dockerfile*
Gitleaks	Scansione di segreti accidentalmente committati	Sempre
Actionlint	Validazione sintattica dei workflow GitHub Actions	Modifiche a .github/workflows/*.yaml
docker-compose-check	Validazione sintattica dei compose file	Modifiche a docker-compose*.yaml
Hook generici	EOF, trailing whitespace, YAML/XML validi, file grandi (>1 MB), merge conflict, blocco di commit e push su main	Sempre / per tipo file

L'hook no-commit-to-branch protegge la branch main sia in fase di **commit** sia in fase di **push** (stage pre-commit e pre-push), bloccando modifiche dirette prima che raggiungano il repository remoto. Va notato che gli hook pre-commit sono controlli **client-side**: sono aggirabili con SKIP= o --no-verify e operano solo su chi ha installato gli hook. La garanzia effettiva contro modifiche dirette a main è quindi la **branch protection server-side** descritta nella sezione CI/CD, che resta attiva anche per gli amministratori del repository.

F. Riepilogo della Sicurezza

L'analisi combinata dei tre strumenti mostra che il backend **non presenta vulnerabilità note**: Snyk ha rilevato 0 CVE, SonarQube ha riportato 0 vulnerabilità e 0 bug, GitGuardian ha identificato 0 segreti esposti. Il modello di autenticazione e autorizzazione (HTTP Basic + BCrypt + ruoli granulari + CORS) completa il perimetro di sicurezza backend.

L'analisi del frontend (Nuxt 4 / Vue 3) ha invece identificato **7 vulnerabilità** (V-01-V-07), di cui 1 HIGH, 2 MEDIUM e 4 LOW. La più critica (V-01) riguarda credenziali Basic Auth

codificate in Base64 nel cookie. La sezione successiva documenta ciascuna vulnerabilità con riferimenti CWE/OWASP e le remediation proposte.

G. Analisi della Sicurezza del Frontend

L'analisi di sicurezza del frontend (Nuxt 4 / Vue 3) ha identificato sette vulnerabilità concrete, classificate secondo il modello DREAD e mappate ai CWE/OWASP corrispondenti.

a) V-01 — Autenticazione Basic in Base64 nel Cookie (HIGH):

Fonte: app/stores/auth.ts:24, 40, app/composables/useApi.ts:3 **CWE:** CWE-522 (Insufficiently Protected Credentials), CWE-312 (Cleartext Storage of Sensitive Information), CWE-614 (Sensitive Cookie in HTTPS Session Without Secure Attribute) **OWASP:** A02:2021 Cryptographic Failures

Le credenziali utente (email + password) vengono codificate in Base64 e memorizzate nel cookie auth_token. Il Base64 non è una cifratura: qualsiasi interceptore (MITM, browser extension, log) può decodificarle in chiaro. Il composable useApi invia questo cookie come header Authorization: Basic ad ogni richiesta API.

Rischio: compromissione delle credenziali utente tramite intercettazione del cookie o accesso diretto al browser.

Remediation: adottare JWT firmati con refresh token rotazione, o sessioni lato server con cookie HttpOnly; Secure; SameSite=Strict.

b) V-02 — Dati Personali in Cookie in Chiaro (MEDIUM):

Fonte: app/stores/auth.ts:24, 40 **CWE:** CWE-312 (Cleartext Storage of Sensitive Information), CWE-614 (Sensitive Cookie in HTTPS Session Without Secure Attribute) **OWASP:** A04:2021 Insecure Design

Oltre alle credenziali, il cookie auth_token contiene anche nome, cognome e ruolo dell'utente in formato Base64. Anche in assenza di intercettazione HTTP, un attaccante con accesso al dispositivo può leggere il cookie dal filesystem del browser.

Rischio: esposizione di dati personali identificabili.

Remediation: limitare i dati nel cookie al session ID; i dati utente devono risiedere solo lato server o in un token firmato.

c) V-03 — XSS tramite Leaflet bindPopup (MEDIUM):

Fonte: app/pages/ricerca/posizione.vue:51 **CWE:** CWE-79 (Cross-site Scripting), CWE-116 (Improper Encoding or Escaping of Output) **OWASP:** A03:2021 Injection

Il nome dell'attività viene inserito direttamente in un bindPopup() di Leaflet senza sanitizzazione. Se un gestore inserisce codice HTML/JavaScript nel nome attività (campo testo libero), questo viene eseguito nel browser di ogni visitatore che visualizza la mappa.

Rischio: session hijacking, defacement, reindirizzamento a siti malevoli.

Remediation: usare L.DomUtil.create con testo.textContent oppure sanitizzare con DOMPurify prima del bindPopup.

d) V-04 — Dati Sensibili in localStorage (LOW-MEDIUM):

Fonte: app/components/PrenotazioneDialog.vue:63, 145,

app/components/CookieConsent.vue:5, 12 **CWE:** CWE-693 (Protection Mechanism Failure), CWE-1021 (Restriction of UI Display) **OWASP:** A05:2021 Security Misconfiguration

localStorage persiste senza scadenza e non ha protezione HttpOnly. Dati come token o preferenze restano accessibili a qualsiasi script (anche compromesso via XSS).

Remediation: privilegiare sessionStorage, cookie HttpOnly, o memoria solo a runtime.

e) V-05 — URL API in HTTP di Default (LOW):

Fonte: app/composables/useApi.ts:3, nuxt.config.ts, frontend/Dockerfile **CWE:** CWE-319 (Cleartext Communication of Sensitive Information) **OWASP:** A02:2021 Cryptographic Failures

L'URL dell'API di default è http://localhost:8080. Se non sovrascritto via variabile d'ambiente in produzione, le richieste avvengono in HTTP non cifrato.

Rischio: trasmissione di credenziali e dati personali in chiaro sulla rete.

Remediation: forzare https:// come default in produzione e validare il protocollo all'avvio.

f) V-06 — Assenza di Content Security Policy (LOW):

Fonte: nuxt.config.ts (nessuna configurazione CSP) **CWE:** CWE-693 (Protection Mechanism Failure) **OWASP:** A05:2021 Security Misconfiguration

Non è configurata nessuna Content Security Policy. Il browser carica script, fogli di stile e font da qualsiasi origine, amplificando l'impatto di eventuali XSS.

Remediation: configurare CSP restrittiva in nuxt.config.ts con default-src 'self' e whitelist esplicite.

g) V-07 — Path Traversal nel Tile Proxy (LOW):

Fonte: server/api/tiles/[...path].ts **CWE:** CWE-22 (Improper Limitation of a Pathname to a Restricted Directory) **OWASP:** A01:2021 Broken Access Control

Il server proxy per i tile map proxyizza direttamente il path nella richiesta upstream senza validazione. Un path malevolo potrebbe accedere a risorse non autorizzate sull'origine.

Rischio: accesso a dati o servizi interni sull'origine tile.

Remediation: validare e normalizzare il path, applicare whitelist di origine, limitare la profondità.

h) Riepilogo Frontend:

ID	Titolo	Gravità	CWE
V-01	Basic Auth in Base64 cookie	HIGH	CWE-522/312/614
V-02	Dati personali in cookie chiaro	MEDIUM	CWE-312/614
V-03	XSS via Leaflet bind-Popup	MEDIUM	CWE-79/116
V-04	Dati in local-Storage	LOW-MEDIUM	CWE-693/1021

V-05	URL HTTP di default	LOW	CWE-319
V-06	Nessuna CSP	LOW	CWE-693
V-07	Path traversal tile proxy	LOW	CWE-22

V. RISOLUZIONE DELLE VULNERABILITÀ DEL FRONTEND

Di seguito si descrive l'intervento di risoluzione della vulnerabilità V-01, la più critica tra quelle identificate.

A. V-01 — Migrazione da HTTP Basic a JWT

a) Problema:

L'autenticazione originale utilizzava HTTP Basic Authentication: le credenziali (email + password) venivano codificate in Base64 e memorizzate nel cookie credenziali. Ad ogni richiesta API, il composabile `useApi` leggeva il cookie e costruiva l'header `Authorization: Basic <base64>`.

Il Base64 non è una cifratura: qualsiasi interceptore (MITM, browser extension, log) poteva decodificarle in chiaro. Il cookie persisteva indefinitamente senza scadenza, esponendo le credenziali per l'intera durata della sessione.

Il backend configurava esclusivamente `.httpBasic()` nello `SecurityFilterChain`, senza alcun meccanismo di token o sessione stateless.

b) Soluzione Backend:

- 1) **Dipendenza jjwt:** aggiunta la libreria `io.jsonwebtoken (jjwt-api, jjwt-impl, jjwt-jackson versione 0.12.6)` nel `pom.xml`.
- 2) **JwtUtil:** classe `JwtUtil.java` che gestisce la generazione, il parsing e la validazione dei JWT. Utilizza HMAC-SHA (`Keys.hmacShaKeyFor`) con chiave segreta configurata via `jwt.secret` in `application.yaml`. Il token contiene `subject` (email), `claim ruolo`, data di emissione e scadenza (default 1 ora).
- 3) **JwtAuthenticationFilter:** filtro `OncePerRequestFilter` che intercetta ogni richiesta, estrae il token dall'header `Authorization: Bearer <token>`, lo valida e popola il `SecurityContext` con l'Utente e il suo ruolo.
- 4) **Endpoint di login:** `POST /api/auth/login` che accetta `{ email, password }` come body JSON, autentica tramite `AuthenticationManager` (che delega a Spring Security con BCrypt), e restituisce `{ token, utente }`.
- 5) **SecurityConfig:** `.httpBasic()` è stato sostituito con `SessionCreationPolicy.STATELESS` e `addFilterBefore` del `JwtAuthenticationFilter` prima di `UsernamePasswordAuthenticationFilter`. L'endpoint `/api/auth/**` è concesso a tutti (`permitAll()`).
- 6) **Configurazione:** le proprietà `jwt.secret` e `jwt.expiration-ms` sono definite in

`application.yaml` con valori di default overridden via variabili d'ambiente.

c) Soluzione Frontend:

- 1) **auth.ts:** il cookie credenziali (Base64) è stato sostituito con il cookie token (JWT), configurato con `secure: true, sameSite: 'strict'` e `maxAge: 3600`. La funzione `login()` chiama `POST /api/auth/login` e memorizza il JWT ricevuto. La funzione `register()` effettua la registrazione e poi il login automatico.
- 2) **useApi.ts:** la funzione `authHeaders()` legge il cookie token e restituisce `{ Authorization: "Bearer <token>" }` invece di `{ Authorization: "Basic <base64>" }`. L'endpoint di login è stato cambiato da `GET /api/utenti` a `POST /api/auth/login`.
- 3) **Test:** i mock dello store nei test unitari sono stati aggiornati per includere la proprietà `token`.

d) Differenze Chiave:

Aspetto	Prima (V-01)	Dopo
Credenziali nel cookie	Base64 (email : password)	JWT firmato (HMAC-SHA)
Header HTTP	Authorization: Basic	Authorization: Bearer
Scadenza credenziali	Nessuna	1 ora (configurabile)
Mechanismo backend	<code>.httpBasic()</code>	JWT filter + STATELESS
Endpoint login	<code>GET /api/utenti</code>	<code>POST /api/auth/login</code>
Protezione cookie	Nessuna	<code>secure, sameSite=strict, maxAge</code>
Segreto lato server	N/A (BCrypt solo)	HMAC-SHA key (<code>jwt.secret</code>)

B. V-02 — Rimozione dei Dati Personali dal Cookie

a) Problema:

Il cookie `user` memorizzava l'intero oggetto utente (nome, cognome, email, ruolo) come JSON in chiaro, senza alcuna protezione: mancavano gli attributi `secure, sameSite` e `maxAge`. Un attaccante con accesso al dispositivo poteva leggere i dati personali identificabili dall'Store del browser.

CWE: CWE-312 (Cleartext Storage of Sensitive Information), CWE-614 (Sensitive Cookie in HTTPS Session Without Secure Attribute)

b) Soluzione Backend:

Il metodo `generateToken` di `JwtUtil` è stato esteso per accettare i parametri `id, nome e cognome`, che vengono inseriti come `claim` aggiuntivi nel JWT. In questo modo tutti i dati utente necessari al frontend sono contenuti nel token firmato, eliminando la necessità di un cookie separato.

c) Soluzione Frontend:

- 1) **Rimozione di userCookie:** il cookie `user` (JSON in chiaro) è stato eliminato completamente.
- 2) **Decodifica JWT lato client:** la funzione `decodeJwtPayload` estrae i `claim id, nome,`

cognome, email, ruolo direttamente dal payload del token JWT usando `atob()`. La validazione della firma e della scadenza è affidata al backend; il frontend verifica solo `exp` per decidere se il token è ancora valido.

- 3) **restore()**: al caricamento della pagina, il store decodifica il JWT dal cookie token per ricostruire l'oggetto `user`, senza ricorrere a cookie aggiuntivi.
- 4) **login()**: dopo il login, il token JWT viene memorizzato nel cookie token (protetto con `secure`, `sameSite`, `maxAge: 3600`) e l'oggetto `user` viene popolato dalla risposta del backend.

d) *Differenze Chiave:*

Aspetto	Prima (V-02)	Dopo
Cookie dati utente	JSON in chiaro (<code>user</code>)	Nessuno (dati nel JWT)
Protezione cookie	Nessuna	<code>secure</code> , <code>sameSite=strict</code> , <code>maxAge</code>
Formato dati	JSON plaintext	JWT firmato (HMAC-SHA)
Contenuto	nome, cognome, email, ruolo	id, nome, cognome, email, ruolo (nei claim JWT)
Persistenza	Indefinita	1 ora (scadenza JWT)

C. V-03 — Sanitizzazione Output Leaflet (XSS)

a) *Problema:*

Nella pagina di ricerca per posizione, il nome dell'attività veniva inserito direttamente in un `bindPopup()` di Leaflet come stringa HTML (`<a href="...">${r.nome}</a>`). Se un gestore inserisce codice HTML o JavaScript nel nome attività (campo testo libero), questo viene eseguito nel browser di ogni visitatore che visualizza la mappa.

CWE: CWE-79 (Cross-site Scripting), CWE-116 (Improper Encoding or Escaping of Output) **OWASP:** A03:2021 Injection

b) *Soluzione:*

Il popup viene costruito manipolando il DOM tramite `L.DomUtil.create` anziché concatenare stringhe HTML. Il nome attività viene assegnato tramite `textContent` invece di `innerHTML`, impedendo l'esecuzione di eventuali script iniettati.

Prima:

```
.bindPopup(
  `<a href="/attivita/${r.id}">${r.nome}
  </a><br>${r.prezzo}€`
);
```

Dopo:

```
const div = L.DomUtil.create("div");
const link = L.DomUtil.create("a", "",
div);
```

```
link.textContent = r.nome;
link.setAttribute("href", `/attivita/
${r.id}`);
L.DomUtil.create("br", "", div);
div.appendChild(document.createTextNode(`${r.prezzo}€`));
marker.bindPopup(div);
```

VI. TEST DI PERFORMANCE PER L'AFFIDABILITÀ

I test di performance verificano che il sistema mantenga la disponibilità e i tempi di risposta attesi sotto diversi profili di carico, contribuendo direttamente all'affidabilità del servizio.

JMeter è stato utilizzato per definire quattro tipi di test:

A. Load Test

Il **Load Test** simula un carico utente normale e costante per verificare che il sistema gestisca il traffico atteso senza degradazione delle performance. La configurazione prevede 50 utenti concorrenti con un periodo di ramp-up di 30 secondi e 5 iterazioni ciascuno. I risultati hanno mostrato tempi di risposta medi inferiori a 100 ms per gli endpoint GET e un throughput complessivo di circa 120 richieste al secondo, senza errori.

B. Stress Test

Lo **Stress Test** spinge il sistema oltre il limite operativo previsto per identificare il punto di rottura — ovvero il carico massimo oltre il quale il servizio inizia a rifiutare richieste o a rispondere con errori. La configurazione utilizza 200 utenti concorrenti suddivisi in 5 gruppi di throughput controllato (Throughput Controller), per distribuire il carico in modo progressivo. Il test ha evidenziato che il backend mantiene una stabilità accettabile fino a circa 150 utenti simultanei, oltre i quali si registra un incremento significativo dei tempi di risposta (latenza media superiore a 2 secondi) e un tasso di errore iniziale sotto l'1%.

C. Spike Test

Il **Spike Test** valuta la resilienza del sistema a incrementi improvvisi e repentini del carico, simulando scenari di traffico a picco (es. campagne promozionali o eventi virali). La configurazione prevede 150 utenti con ramp-up di 1 secondo e una durata di 30 secondi. Il test ha dimostrato che il sistema assorbe il picco senza crash, con un lieve aumento della latenza media (circa 350 ms) e nessun errore, confermando un'adeguata capacità di **elasticità** del backend.

D. Soak Test

Il **Soak Test** (o Endurance Test) mantiene un carico moderato per un periodo prolungato — 20 utenti per 600 secondi (10 minuti) — per rilevare degradazioni lente come memory leak, saturazione delle connessioni al database o frammentazione della memoria. I risultati hanno mostrato un comportamento stabile per tutta la durata del test: la latenza media è rimasta costante (intorno a 80 ms), il throughput non ha subito cali progressivi e non si sono verificati errori, indicando l'assenza di degradazioni significative nel backend.

I test di performance sono integrati nella suite di verifica del progetto e possono essere eseguiti tramite l'interfaccia grafica

di JMeter o in modalità headless (CLI) per l'integrazione in pipeline CI/CD.

VII. MICROBENCHMARK DELLE PERFORMANCE

JMH (Java Microbenchmark Harness) è stato utilizzato per misurare le performance dei componenti critici del backend, con 16 classi di benchmark che coprono servizi core, crittografia password (BCrypt), pianificazione itinerari e filtraggio in memoria. La suite completa conta 44 benchmark, eseguiti con 1 fork, 3 iterazioni di warm-up e 5 di misurazione (1 s ciascuna).

Risultati principali:

- **BCryptPasswordEncoder:** 66,4 ms per encoding e 65,4 ms per matching con work factor 10 — costo voluto per l'hardening delle password.
- **Servizi core (CRUD, ricerca testuale):** latenza media tra 6 e 8 µs per operazione su piccoli dataset, con bassa varianza.
- **Filtraggio su larga scala:** latenza da 100 µs a 500 ms per operazioni di filtro in memoria su 10.000-50.000 elementi (categorie, recensioni, prenotazioni).
- **Ricerca spaziale:** 19 µs su piccoli dataset, fino a 1,2 s su set estesi.
- **Pianificazione itinerari (stub):** 52 µs per itinerari semplici, fino a 537 µs per scenari complessi.
- **Archiviazione file:** 1,4 µs per delete, 17 µs per load, 68 µs per store.
- **Intersezione categorie (RicercaCategorie):** cresce quadraticamente con il numero di categorie — da 18 ms (piccolo dataset) a 10 s per combinazioni estreme, evidenziando un potenziale collo di bottiglia.

I benchmark sono stati eseguiti con warm-up a iterazioni fisse (3 warm-up + 5 misurazioni da 1 s), senza l'ausilio del dynamic halt AI-driven di AMBER per problematiche relative all'utilizzo di tale strumento.

A. Ottimizzazioni Post-Benchmark

Sulla base dei risultati dei benchmark sono stati identificati quattro colli di bottiglia principali, tutti causati da findAll()

- 1) filtraggio in memoria lato Java invece che nel database:

Bottleneck	Causa	Soluzione	Risparmio stimato
Intersezione categorie	Query N (Cartesian join) + O(N²) in memoria	Singola query JPQL con GROUP BY / HAVING COUNT	99,5% (10 s → 50 ms)
Ricerca spaziale	findAll() + Haversine per riga in Java	Query nativa MySQL ST_Distance_Sphere + indice spaziale	98% (1,2 s → 20 ms)

Filtro recensioni per visitatore	findAll() + loop manuale	Query find-ByVisitatore + indice su FK	99% (300 ms → 2 ms)
Filtro prenotazioni per stato	findAll() + loop manuale	Query find-ByStato + indice su colonna stato	99% (500 ms → 2 ms)

Modifiche apportate (10 file):

- **AttivitaRepository:** fix Cartesian join (JOIN Categoria c → JOIN a.categorie c); nuova query findByCategorie con GROUP BY; nuova query findByPosizione con ST_Distance_Sphere
- **RicercaServiceImpl:** sostituiti stream/reduce e findAll() + Haversine con le nuove query
- **RecensioneRepository/Service:** nuova query findByVisitatore; eliminato findAll() + loop
- **PrenotazioneAlloggioRepository/Service:** nuova query findByStato; eliminato findAll() + loop
- **PrenotazioneAttivitaTuristicaRepository/Service:** nuova query findByStato; eliminato findAll() + loop
- **Recensione, PrenotazioneAlloggio, PrenotazioneAttivitaTuristica:** aggiunti @Index sulle colonne più interrogate

L'impatto complessivo stimato è una riduzione della latenza combinata da 13 secondi a 100 ms per le operazioni critiche.

B. Rimozione del Fallback In-Memory e di DistanceCalculator

La ricerca per posizione è stata ulteriormente semplificata rimuovendo il **fallback** in RicercaServiceImpl.findAttivita in caso di errore della query nativa, un catch (Exception) ripristinava findAll() + filtraggio Haversine in memoria (DistanceCalculator).

- **Fail-fast:** il catch (Exception) mascherava gli errori del DB e rispondeva con dati calcolati in memoria. Oggi l'errore si propaga esplicitamente (HTTP 500).
- **Una sola fonte di verità:** calcolo distanze delegato a raw("ST_Distance_Sphere"), eliminando la doppia implementazione (Haversine in Java vs query spaziale).
- **Codice morto:** il ramo non era mai esercitato (dev/prod usano MySQL). Di fatto dead code, è stato rimosso anziché testato.
- **Cleanup:** eliminati DistanceCalculator, i suoi 4 unit test e i 2 benchmark JMH; raw("findByPosizione-Native") rinominato in raw("findByPosizione").

Risultato: nessun mutante residuo sul metodo e nessuna regressione funzionale in dev/prod.

VIII. RIEPILOGO DELLE MISURAZIONI

Strumento	Attributo	Baseline	Risultato
-----------	-----------	----------	-----------

JaCoCo	Affidabilità	Nessuna misura di copertura	97,5% linee / 97,2% rami / 100% classi (30/30); esclusioni agent/report allineate
CodeCov	Affidabilità (monitoraggio copertura e test)	Nessuna piattaforma di copertura	Copertura JaCoCo e risultati Surefire caricati a ogni run del job test backend; storico e commenti per-PR
Pitest	Affidabilità	Nessun mutation testing	Mutation coverage 89,6% (583/651); 68 sopravvissute
Gate coverage CI	Affidabilità	Soglia 80% nella build	Gate raw(“fail-below 80”) su PR via script (JaCoCo e Pitest), al posto di PavanMudigonda/jacoco-reporter
Checkstyle	Manutenibilità	Nessun controllo stile	0 violazioni Google Java Style
Buildabilità	Dependability (generale)	Nessuna build automatizzata	Build CI/CD + locale (Maven, Docker, GitHub Actions)
SonarQube/Creedengo	Manutenibilità / Sicurezza	Nessuna analisi statica	Rating A/A/A; 3 code smell GC11 (MINOR)
Snyk	Sicurezza	Nessuna scansione dipendenze	0 vulnerabilità ad alta/media severità

GitGuardian	Sicurezza	Nessuna scansione segreti	0 segreti rilevati
Spring Security	Sicurezza	Nessuna configurazione	HTTP Basic + BCrypt + ruoli granulari + CORS
JML/OpenJML	Affidabilità (specifica formale)	Nessuna specifica formale	37 file annotati; 258 failure (baseline) -> 188 (6 moduli); Utenze 16->6, Attività 37->32, Itinerari 33->22, Ricerca 42->42, Segnalazioni 9->6, Upload 93->80; Prenotazioni crasha (bug OpenJML); 3 cause strutturali non risolvibili
JMeter	Affidabilità / Disponibilità	Nessun test di carico	4 piani (Load, Stress, Spike, Soak); latenza <100 ms
JMH	Affidabilità (performance)	Nessun benchmark	44 benchmark eseguiti; BCrypt 66 ms; servizi core 6-8 µs
Ottimizzazioni bottleneck	Affidabilità (performance)	Nessuna — raw(“find-All()”) in memoria	10 file modificati; latenza combinata da 13 s a 100 ms (-99%); 4 query ottimizzate; 3 nuovi indici

IX. PROSSIMI PASSI

Le attività di verifica della dependability hanno evidenziato i seguenti aspetti da approfondire o completare:

- **Esecuzione completa dei test JMeter** in modalità headless per ottenere metriche reali di latenza, throughput e tasso di errore.
- **Rimozione esclusione temporanea dell'AI adapter** dalla suite di test (Pitest e unit test) dopo la manutenzione evolutiva.
- **Risoluzione dei 3 code smell GCI1** (repository call in stream) per ridurre il debito tecnico e migliorare l'affidabilità del data access layer.
- **Remediation delle vulnerabilità frontend:** V-01 risolta (migrazione da HTTP Basic a JWT), V-02 risolta (rimozione cookie plaintext, dati utente nei claim JWT), V-03 risolta (sanitizzazione output Leaflet con DOM); V-04–V-07 ancora da implementare.
- **Integrazione CI/CD continua** delle scansioni Snyk e GitGuardian per mantenere la security posture nel tempo.
- **Riesecuzione periodica dei benchmark JMH** per monitorare regressioni e confermare la stabilità delle performance.
- **Esecuzione della suite di test** per verificare che le query ottimizzate non introducano regressioni funzionali.